

Banking System Design with PostgreSQL

Table of Contents

1. [Overview](#)
 2. [Requirements](#)
 3. [Capacity Estimation](#)
 4. [Schema Design](#)
 5. [ASCII ER Diagram](#)
 6. [Indexing Strategy](#)
 7. [Query Patterns](#)
 8. [Scaling Strategy](#)
 9. [Tradeoffs](#)
 10. [Interview Discussion Points](#)
 11. [Common Interview Follow-ups](#)
 12. [Performance Considerations](#)
 13. [Cross-References](#)
-

Overview

Banking is where correctness is non-negotiable. Unlike most internet systems where "eventually consistent" is acceptable, banking requires **immediate consistency** for account balances and transactions. This document covers the full system design: account management, transaction processing, ledger maintenance, isolation level selection, and the exact ACID guarantees required at each stage. It builds on the schema from `18_Architecture_Case_Studies/04_banking_platform.md` and focuses on system architecture and consistency reasoning.

Requirements

Functional Requirements

- Account management: open, close, deposit, withdraw
- Transfers: same-bank (internal) and inter-bank (ACH, SWIFT, SEPA)
- Balance inquiry: current balance, available balance (accounting for holds)
- Transaction history: paginated statement
- Recurring payments: standing orders, direct debits
- Cards: debit/credit, authorization and settlement
- Interest calculation: daily accrual, monthly posting
- Dispute resolution: chargeback workflow

Non-Functional Requirements

- Durability: zero data loss (`fsync=on` , synchronous replication)
 - Consistency: account balances always correct; no money created or lost
 - Isolation: `SERIALIZABLE` for transfers; `READ COMMITTED` for balance inquiry
 - Availability: 99.99% for transfers; scheduled maintenance windows allowed
 - Auditability: every state change logged with actor, timestamp, previous value
 - Compliance: 7-year transaction retention; PCI-DSS; AML/KYC
-

Capacity Estimation

Metric	Value
Customers	50M
Accounts	125M (2.5 avg)
Transactions per day	500M
Peak TPS (transactions/sec)	10K
Ledger entries per day	1B (2 per txn)
Fraud checks per transaction	1
ACH batch items per day	50M

Storage Estimates (7-year retention)

Table	Rows	Row Size	Total
accounts	125M	600B	75GB
transactions	1.3T	700B	910TB
ledger_entries	2.6T	350B	910TB
audit_log	4T	500B	2PB

Observation: 7-year audit log is 2PB. This cannot live in a single PostgreSQL cluster. The solution is tiered storage:

- **Hot:** Last 6 months in PostgreSQL (partitioned quarterly) — ~100TB
- **Warm:** 6 months – 3 years in columnar storage (Redshift/BigQuery) — ~500TB
- **Cold:** 3–7 years in immutable object storage (S3 with Object Lock) — ~1.5PB

Schema Design

```
-- (Full schema in 18_Architecture_Case_Studies/04_banking_platform.md)
-- This section focuses on the transaction processing and consistency core.

CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- =====
-- ACCOUNTS (Balance as Single Source of Truth)
-- =====
CREATE TYPE account_type AS ENUM ('checking', 'savings', 'credit', 'loan');
CREATE TYPE account_status AS ENUM ('active', 'dormant', 'frozen', 'closed');

CREATE TABLE accounts (
    account_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```

account_number    VARCHAR(30)    NOT NULL,
customer_id       UUID          NOT NULL,
account_type      account_type  NOT NULL,
status            account_status NOT NULL DEFAULT 'active',
currency          CHAR(3)       NOT NULL DEFAULT 'USD',
-- Balance columns
current_balance   NUMERIC(19,4) NOT NULL DEFAULT 0,
available_balance NUMERIC(19,4) NOT NULL DEFAULT 0,
-- Constraints
overdraft_limit   NUMERIC(19,4) NOT NULL DEFAULT 0,
credit_limit      NUMERIC(19,4),
-- Metadata
opened_at         TIMESTAMPTZ   NOT NULL DEFAULT now(),
closed_at         TIMESTAMPTZ,
last_txn_at       TIMESTAMPTZ,
CONSTRAINT uq_account_number UNIQUE (account_number),
-- Balance floor considering overdraft
CONSTRAINT chk_balance_floor
    CHECK (current_balance >= -overdraft_limit)
);

-- =====
-- TRANSACTIONS (Immutable)
-- =====
-- INVARIANT: Once status = 'completed', the record NEVER changes.
-- Corrections use reversal transactions (new rows).

CREATE TYPE txn_type AS ENUM (
    'deposit', 'withdrawal', 'internal_transfer_in', 'internal_transfer_out',
    'ach_credit', 'ach_debit', 'wire_in', 'wire_out',
    'card_purchase', 'card_refund', 'interest', 'fee', 'adjustment', 'reversal'
);

CREATE TYPE txn_status AS ENUM ('pending', 'processing', 'completed', 'failed',
    'reversed');

CREATE TABLE transactions (
    txn_id          UUID          NOT NULL DEFAULT uuid_generate_v4(),
    txn_reference    VARCHAR(50)   NOT NULL,
    txn_type         txn_type      NOT NULL,
    status           txn_status    NOT NULL DEFAULT 'pending',
    amount          NUMERIC(19,4) NOT NULL,
    currency         CHAR(3)       NOT NULL,
    description      VARCHAR(500),
    idempotency_key  VARCHAR(100), -- client-provided, prevents double-
processing
    reversal_of      UUID,         -- points to original txn if this is a
reversal
    initiated_at     TIMESTAMPTZ   NOT NULL DEFAULT now(),
    processed_at     TIMESTAMPTZ,
    value_date       DATE,
    metadata         JSONB,

```

```

    PRIMARY KEY (txn_id, initiated_at),
    CONSTRAINT uq_txn_reference UNIQUE (txn_reference),
    CONSTRAINT uq_idempotency UNIQUE (idempotency_key),
    CONSTRAINT chk_amount_positive CHECK (amount > 0)
) PARTITION BY RANGE (initiated_at);

-- Quarterly partitions
CREATE TABLE transactions_2024_q1 PARTITION OF transactions
    FOR VALUES FROM ('2024-01-01') TO ('2024-04-01');
CREATE TABLE transactions_2024_q2 PARTITION OF transactions
    FOR VALUES FROM ('2024-04-01') TO ('2024-07-01');

-- =====
-- DOUBLE-ENTRY LEDGER
-- =====
-- INVARIANT: For every transaction, SUM(debit amounts) = SUM(credit amounts)
-- INVARIANT: Every ledger entry is immutable once created.

CREATE TYPE entry_side AS ENUM ('debit', 'credit');

CREATE TABLE ledger_entries (
    entry_id          UUID          NOT NULL DEFAULT uuid_generate_v4(),
    txn_id            UUID          NOT NULL,
    account_id        UUID          NOT NULL REFERENCES accounts(account_id),
    side              entry_side    NOT NULL,
    amount            NUMERIC(19,4) NOT NULL,
    currency          CHAR(3)       NOT NULL,
    running_balance   NUMERIC(19,4) NOT NULL, -- balance AFTER this entry
    entry_date        DATE          NOT NULL DEFAULT CURRENT_DATE,
    value_date        DATE,
    PRIMARY KEY (entry_id, entry_date),
    CONSTRAINT chk_entry_amount CHECK (amount > 0)
) PARTITION BY RANGE (entry_date);

-- =====
-- HOLDS (Pending Charges That Reduce Available Balance)
-- =====
CREATE TABLE holds (
    hold_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    account_id       UUID          NOT NULL REFERENCES accounts(account_id),
    amount           NUMERIC(19,4) NOT NULL,
    currency         CHAR(3)       NOT NULL,
    reason           VARCHAR(200),
    expires_at       TIMESTAMPTZ   NOT NULL,
    released_at      TIMESTAMPTZ,
    is_active        BOOLEAN       NOT NULL DEFAULT TRUE,
    created_at       TIMESTAMPTZ   NOT NULL DEFAULT now()
);

-- =====
-- THE CORE TRANSFER FUNCTION
-- =====

```

```

CREATE OR REPLACE FUNCTION internal_transfer(
    p_from_account_id    UUID,
    p_to_account_id      UUID,
    p_amount             NUMERIC,
    p_currency            CHAR(3),
    p_description         VARCHAR,
    p_idempotency_key    VARCHAR
) RETURNS TABLE(txn_id UUID, status TEXT) AS $$
DECLARE
    v_txn_id             UUID;
    v_from_balance       NUMERIC;
    v_to_balance         NUMERIC;
    v_ref                VARCHAR;
    v_exists             UUID;
BEGIN
    -- IDEMPOTENCY CHECK: prevent double-processing on retries
    SELECT t.txn_id INTO v_exists
    FROM transactions t
    WHERE t.idempotency_key = p_idempotency_key;

    IF FOUND THEN
        RETURN QUERY SELECT v_exists, 'already_processed'::TEXT;
        RETURN;
    END IF;

    -- LOCK ACCOUNTS in deterministic order (prevents deadlock)
    -- Always lock lower UUID first
    IF p_from_account_id < p_to_account_id THEN
        SELECT a.current_balance INTO v_from_balance
        FROM accounts a WHERE a.account_id = p_from_account_id FOR UPDATE;
        PERFORM FROM accounts WHERE account_id = p_to_account_id FOR UPDATE;
    ELSE
        PERFORM FROM accounts WHERE account_id = p_to_account_id FOR UPDATE;
        SELECT a.current_balance INTO v_from_balance
        FROM accounts a WHERE a.account_id = p_from_account_id FOR UPDATE;
    END IF;

    -- SUFFICIENT FUNDS CHECK
    IF v_from_balance < p_amount THEN
        RAISE EXCEPTION 'insufficient_funds: balance=%, required=%',
            v_from_balance, p_amount
            USING ERRCODE = 'P0001';
    END IF;

    -- GENERATE REFERENCES
    v_txn_id := uuid_generate_v4();
    v_ref := 'TXN-' || TO_CHAR(now(), 'YYYYMMDD') || '-' ||
upper(substr(v_txn_id::text, 1, 8));

    -- CREATE TRANSACTION RECORD
    INSERT INTO transactions
        (txn_id, txn_reference, txn_type, status, amount, currency, description,

```

```

idempotency_key, processed_at)
VALUES
    (v_txn_id, v_ref, 'internal_transfer_out', 'completed',
     p_amount, p_currency, p_description, p_idempotency_key, now());

-- DEBIT SOURCE ACCOUNT
UPDATE accounts
SET current_balance      = current_balance - p_amount,
    available_balance    = available_balance - p_amount,
    last_txn_at          = now()
WHERE account_id = p_from_account_id;

INSERT INTO ledger_entries
    (txn_id, account_id, side, amount, currency, running_balance)
SELECT v_txn_id, p_from_account_id, 'debit', p_amount, p_currency,
current_balance
FROM accounts WHERE account_id = p_from_account_id;

-- CREDIT DESTINATION ACCOUNT
UPDATE accounts
SET current_balance      = current_balance + p_amount,
    available_balance    = available_balance + p_amount,
    last_txn_at          = now()
WHERE account_id = p_to_account_id;

INSERT INTO ledger_entries
    (txn_id, account_id, side, amount, currency, running_balance)
SELECT v_txn_id, p_to_account_id, 'credit', p_amount, p_currency,
current_balance
FROM accounts WHERE account_id = p_to_account_id;

RETURN QUERY SELECT v_txn_id, 'completed'::TEXT;
END;
$$ LANGUAGE plpgsql;

```

ASCII ER Diagram

```

+-----+ +-----+ +-----+
| customers|1----N| accounts |1----N| transactions |
+-----+ +-----+ +-----+
                |1                |1
                |N                |N
                +-----+ +-----+
                | holds | |ledger_entries |
                +-----+ +-----+

+-----+ +-----+
| accounts |1----N| balance |
+-----+ |snapshots |
                +-----+

```

```
+-----+ +-----+
| transactions | 1-----N | audit_log |
+-----+ +-----+
(via trigger)
```

Indexing Strategy

```
-- Accounts: customer lookup and number/IBAN lookup
CREATE INDEX idx_accounts_customer ON accounts (customer_id, account_type);
CREATE INDEX idx_accounts_number   ON accounts (account_number);

-- Transactions: statement generation (most common query)
CREATE INDEX idx_txns_date          ON transactions (initiated_at DESC);
CREATE INDEX idx_txns_reference     ON transactions (txn_reference);
CREATE INDEX idx_txns_idempotency   ON transactions (idempotency_key)
    WHERE idempotency_key IS NOT NULL;

-- Ledger: account statement (hot path)
CREATE INDEX idx_ledger_account     ON ledger_entries (account_id, entry_date DESC);
CREATE INDEX idx_ledger_txn         ON ledger_entries (txn_id);

-- Holds: active holds affect available_balance
CREATE INDEX idx_holds_active       ON holds (account_id, expires_at)
    WHERE is_active = TRUE;

-- Balance snapshots: point-in-time query
CREATE INDEX idx_snapshots_account ON account_balance_snapshots
    (account_id, snapshot_date DESC);
```

Query Patterns

Q1 — Account Statement

```
SELECT
    le.entry_date,
    le.value_date,
    t.txn_reference,
    t.txn_type,
    t.description,
    le.side,
    le.amount,
    le.running_balance
FROM ledger_entries le
JOIN transactions t ON t.txn_id = le.txn_id
WHERE le.account_id = $1
    AND le.entry_date BETWEEN $from AND $to
```

```
ORDER BY le.entry_date DESC, le.entry_id DESC
LIMIT 100 OFFSET $offset;
```

Q2 — Execute Internal Transfer

```
-- Application sets isolation level before calling
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

SELECT txn_id, status
FROM internal_transfer(
    p_from_account_id => $from_account::UUID,
    p_to_account_id   => $to_account::UUID,
    p_amount          => $amount::NUMERIC,
    p_currency        => $currency::CHAR(3),
    p_description     => $desc,
    p_idempotency_key => $idempotency_key
);
```

Q3 — Current Account Position

```
SELECT
    a.account_id,
    a.account_number,
    a.account_type,
    a.current_balance,
    a.available_balance,
    -- Total active holds reducing available balance
    COALESCE(
        (SELECT SUM(h.amount)
         FROM holds h
         WHERE h.account_id = a.account_id AND h.is_active AND h.expires_at >
now()),
        0
    ) AS total_holds
FROM accounts a
WHERE a.account_id = $1;
```

Q4 — Reconciliation: Verify Ledger Balance

```
-- Every debit must equal every credit for a completed transaction
SELECT
    t.txn_id, t.txn_reference,
    SUM(le.amount) FILTER (WHERE le.side = 'debit') AS total_debits,
    SUM(le.amount) FILTER (WHERE le.side = 'credit') AS total_credits,
    ABS(
        SUM(le.amount) FILTER (WHERE le.side = 'debit') -
        SUM(le.amount) FILTER (WHERE le.side = 'credit')
    ) AS imbalance
```



```

FROM transactions t
JOIN ledger_entries le ON le.txn_id = t.txn_id
WHERE t.initiated_at::date = $date
      AND t.status = 'completed'
GROUP BY t.txn_id, t.txn_reference
HAVING ABS(
    SUM(le.amount) FILTER (WHERE le.side = 'debit') -
    SUM(le.amount) FILTER (WHERE le.side = 'credit')
) > 0.001
ORDER BY imbalance DESC;
-- Should return 0 rows. Any row = critical integrity issue.

```

Q5 — Daily Batch: Process Standing Orders

```

SELECT
    so.so_id, so.from_account_id, so.to_account_id,
    so.amount, so.currency, so.description
FROM standing_orders so
WHERE so.next_execution = CURRENT_DATE
      AND so.is_active
      AND (so.end_date IS NULL OR so.end_date >= CURRENT_DATE)
ORDER BY so.from_account_id -- process same account together
LIMIT 1000;
-- Application: call internal_transfer() for each, update next_execution

```

Q6 — Fraud Velocity Check

```

-- Check last 1 hour: count and sum
SELECT
    COUNT(*) AS txn_count_1h,
    SUM(t.amount) AS total_amount_1h
FROM transactions t
JOIN ledger_entries le ON le.txn_id = t.txn_id
WHERE le.account_id = $account_id
      AND t.initiated_at >= now() - INTERVAL '1 hour'
      AND t.status != 'failed';

```

Q7 — Interest Posting (month-end batch)

```

BEGIN;
-- Calculate interest per savings account
WITH interest_calc AS (
    SELECT
        a.account_id,
        a.current_balance * a.interest_rate / 12 AS interest_amount
    FROM accounts a
    WHERE a.account_type = 'savings'
          AND a.status = 'active'

```

```

        AND a.current_balance > 0
    ),
    -- Insert one transaction per account
    new_txns AS (
        INSERT INTO transactions (txn_id, txn_reference, txn_type, status, amount,
        currency, description, processed_at)
        SELECT
            uuid_generate_v4(),
            'INT-' || TO_CHAR(now(), 'YYYYMM') || '-' || account_id,
            'interest',
            'completed',
            interest_amount,
            'USD',
            'Monthly interest',
            now()
        FROM interest_calc
        RETURNING txn_id, (metadata->>'account_id')::UUID AS account_id
    )
    -- Credit each account and add ledger entry
    UPDATE accounts a
    SET current_balance      = current_balance + ic.interest_amount,
        available_balance    = available_balance + ic.interest_amount,
        last_txn_at          = now()
    FROM interest_calc ic
    WHERE a.account_id = ic.account_id;

    COMMIT;

```

Q8 — Account Closure Check

```

-- Account can only be closed if balance is 0 and no pending transactions
SELECT
    a.current_balance,
    a.available_balance,
    EXISTS (
        SELECT 1 FROM transactions t
        JOIN ledger_entries le ON le.txn_id = t.txn_id
        WHERE le.account_id = $1
        AND t.status IN ('pending', 'processing')
    ) AS has_pending_txns,
    EXISTS (
        SELECT 1 FROM holds h
        WHERE h.account_id = $1 AND h.is_active AND h.expires_at > now()
    ) AS has_active_holds
FROM accounts a
WHERE a.account_id = $1;

```

Q9 — Balance at Historical Date

```

-- First try pre-computed snapshot
SELECT balance
FROM account_balance_snapshots
WHERE account_id = $1
      AND snapshot_date <= $date
ORDER BY snapshot_date DESC
LIMIT 1;

-- If no snapshot, compute from ledger
SELECT running_balance
FROM ledger_entries
WHERE account_id = $1
      AND entry_date <= $date
ORDER BY entry_date DESC, entry_id DESC
LIMIT 1;

```

Q10 — High-Value Transaction Report (CTR)

```

SELECT
    c.full_name, a.account_number,
    t.txn_reference, t.txn_type,
    t.amount, t.currency, t.initiated_at,
    t.metadata->>'merchant_name' AS merchant
FROM transactions t
JOIN ledger_entries le ON le.txn_id = t.txn_id
JOIN accounts a ON a.account_id = le.account_id
JOIN customers c ON c.customer_id = a.customer_id
WHERE t.initiated_at::date = $report_date
      AND t.amount >= 10000
      AND t.status = 'completed'
ORDER BY t.amount DESC;

```

Scaling Strategy

Principle: Correctness Constrains Scaling Choices

Every scaling decision must preserve ACID guarantees for transfers. Options that sacrifice consistency (async writes, eventual consistency) are off the table for the core ledger.

Phase 1: Single Primary + Synchronous Replica (0–10M customers)

```

Client → Load Balancer → App Servers (stateless)
                        |
                        PgBouncer (transaction mode)
                        |
                        Primary PostgreSQL (writes)
                        |
                        Synchronous Replica (synchronous_commit=remote_apply)

```

|
Async Replica (reporting, analytics)

- `synchronous_commit = remote_apply` : Confirms write only after replica has applied it. Protects against primary failure causing data loss.
- Separate read replica for non-transactional reads (balance inquiries, statements).

Phase 2: Functional Decomposition (10M–100M customers)

- **CoreBankingDB**: accounts, ledger, transactions (requires `SERIALIZABLE`)
- **CRMDB**: customers, KYC, addresses (`READ COMMITTED` fine)
- **FraudDB**: fraud rules, alerts, velocity (`READ COMMITTED`, near-real-time)
- **AuditDB**: audit log, regulatory reports (append-only, async writes from trigger)
- **AnalyticsDB**: read replica or Redshift for reporting

Phase 3: Geographic Distribution (100M+ customers)

- **Active-Passive per region**: Writes go to regional primary; global primary replicates to all regions.
- **Account sharding**: Shard accounts by `hash(account_id) % N`. All transactions for accounts on the same shard are local.
- **Cross-shard transfers**: Use a distributed saga with compensating transactions. A two-phase commit coordinator (or XA transactions via PgJDBC) handles atomicity across shards.
- **Partitioning**: Transactions partitioned quarterly. Partitions older than 2 years archived to immutable cold storage.

Isolation Level Selection by Operation

Operation	Isolation Level	Reason
Fund transfer	<code>SERIALIZABLE</code>	Prevents write skew (concurrent overdraft)
Balance inquiry	<code>READ COMMITTED</code>	Current balance, no phantom concern
Statement generation	<code>REPEATABLE READ</code>	Consistent snapshot for the statement
Fraud rule evaluation	<code>READ COMMITTED</code>	Speed matters; slight lag acceptable
Interest calculation batch	<code>REPEATABLE READ</code>	Consistent view of all account balances
Audit log read	<code>READ COMMITTED</code>	Read-only report, no consistency issues

Tradeoffs

Decision	Chosen	Alternative	Why
<code>SERIALIZABLE</code> for transfers	<code>SERIALIZABLE</code>	<code>REPEATABLE READ</code> + advisory locks	<code>SERIALIZABLE</code> catches write skew (overdraft race); SSI in PG is efficient
<code>synchronous_commit=remote_apply</code>	Two replicas confirmed	async commit	Protects against primary crash between commit and replica sync; slight latency increase

Immutable transactions	No UPDATE on completed	Soft delete + version	Standard accounting; corrections via reversal entries; never rewrite history
Double-entry ledger	Two entries per txn	Single balance update	Mathematically verifiable; self-auditing; standard banking practice for 500+ years
Row-level locking for transfers	SELECT FOR UPDATE	Optimistic with retry	At banking scale (not Twitter scale), pessimistic locking is simpler and sufficient

Interview Discussion Points

- 1. Why SERIALIZABLE instead of REPEATABLE READ?** Consider two concurrent transactions both checking if Account A has \$500 to cover a \$400 transfer. Both see \$500 balance (REPEATABLE READ guarantees they see the same snapshot). Both proceed to deduct \$400. Result: Account A balance = \$-300 (write skew). SERIALIZABLE's SSI detects the read-write conflict and aborts one transaction.
- 2. How do you prevent a double payment?** The `idempotency_key` column on transactions has a UNIQUE constraint. The client generates a UUID for the payment attempt and includes it in every retry. If the server receives the same key twice, the second request returns the existing transaction rather than creating a new one.
- 3. What happens if the primary dies during a transfer?** With `synchronous_commit = remote_apply`, the transaction was not acknowledged to the client until the replica applied it. The replica has the complete, consistent state. Failover promotes the replica; no data loss; in-progress uncommitted transactions are rolled back and the client retries.
- 4. How does the double-entry ledger enable fraud detection?** Every legitimate transaction must have debits equal credits. A fraud detection job runs Q4 (reconciliation) nightly. Any imbalance indicates corruption or fraud. Additionally, velocity rules query `ledger_entries` to detect unusual debit patterns per account over rolling time windows.

Common Interview Follow-ups

Q: How do you handle a payment that is processed twice (network timeout, client retries)? A: The idempotency key on `transactions` with a UNIQUE constraint is the primary defense. The `internal_transfer` function checks for an existing transaction with the same key before executing. The function is wrapped in the caller's transaction; if the INSERT of the transaction fails with a duplicate key violation, the function returns the existing result.

Q: How do you scale ACH batch processing (50M items/day)? A: ACH items arrive as a file (NACHA format). A batch processor reads the file, validates each item, and uses PostgreSQL `COPY` to bulk-insert into a staging table. A worker pool processes items from the staging table using `SELECT FOR UPDATE SKIP LOCKED`, executing `internal_transfer` for each. At 50M items/day = ~580/second — manageable on a single PostgreSQL primary with connection pooling.

Q: How do you handle currency conversion in transfers? A: The `transfers` table stores `fx_rate` (the rate applied) and `to_amount` (amount in destination currency). The FX rate is fetched from an external provider at transfer time and stored immutably. The debit (from account) and credit (to account) are in their respective currencies; the ledger stores each in its own currency. Monthly reports convert to a reporting currency using end-of-month rates.

Performance Considerations

- **HOT updates on accounts:** `current_balance` is updated on every transaction. Set `fillfactor=80` on the `accounts` table to enable HOT updates (in-place updates that don't need index pointer updates).
 - **SERIALIZABLE throughput:** PostgreSQL SSI can sustain ~10K serializable transactions/second per node on modern hardware. Above this, shard by account.
 - **Partition pruning:** Transaction queries always include `initiated_at` bounds. Without partition key in WHERE, PostgreSQL scans all quarterly partitions (potentially 28 partitions for 7 years of data).
 - **Prepared statements:** The `internal_transfer` function is called thousands of times per second. Use prepared statements at the connection pool level to eliminate parse overhead.
 - **VACUUM frequency:** The `accounts` table has massive UPDATE churn. Set `autovacuum_vacuum_scale_factor = 0.01` (1% dead tuples triggers vacuum vs. default 20%).
-

Cross-References

- See `18_Architecture_Case_Studies/04_banking_platform.md` for full schema
- See `02_ecommerce_system_design.md` for payment saga patterns
- See `08_system_design_framework.md` for interview presentation framework
- Isolation levels: Chapter 6 of this guide
- ACID transactions: Chapter 4 of this guide
- Partitioning: Chapter 5 of this guide